

Topic 3.6

3.6: More on Input and Output

When using *stream extraction* operators, white space is a delimiter. This is usually desirable when obtaining numeric inputs, but often creates problems when attempting to input character and string inputs.

There are a number of other functions which can be applied against an *istream*. They are described briefly. The standard input stream `cin` is used, but these operations can be applied against any *istream* object. Here is a brief description of these functions:

`cin.get()` obtains the next character, returning it as an `int`. The behavior is similar to the `getchar()` function from `stdio.h`.

`cin.get(a)` obtains the next character and stores it in the `char` argument `a`. The function also returns a value which is a reference to the accessed stream. This allows cascading:

```
cin.get(a) >> x;  
cin.get(a).get(b).get(c);
```

`cin.get( char_array, max_chars, terminator_char )` reads characters into `char_array` until `max_chars` are read, or until `terminator_char` is found. `terminator_char` is not read.

`cin.getline( char_array, max_chars )` reads characters into `char_array` until `max_chars` are read, or until `newline` is read.

`cin.ignore( max_chars, terminator_char )` skips characters until `max_chars` chars have been skipped or until `terminator_char` is read and ignored.

`cin.peek()` returns the next character but leaves it available for the next input operation.

`cin.putback( one_char )` puts `one_char` back into the stream making it available as the next input character.

The identifier `endl` outputs a newline and also causes the output buffer to be flushed.

`cout.put( one_char )` outputs a single character.

`cout.write( char_array, nchars )` outputs `nchars` chars from `char_array`.

`cout.width(ncols)` sets the output fieldwidth for the next field only to `ncols`.

`cout.setf` is used to control the base of numbers output, leading sign, number of decimal digits, etc.

This shows how to use a stream to access files:

```
#include <fstream.h>

ifstream data_in("data.in");
ofstream data_out("data.out");

data_in >> x;
data_out << x;

data_out.close();
```

The following small program illustrates how to use *ifstream*s and *ofstream*s to access disk files:

```
// FILENAME: adder.cpp
// PROGRAMMER: Gary J. Blair
// DATE: 4/11/1996
// COMPILER: MS Visual C++ 2008 Express Edition
// REQUIRED: adder.cpp
// PURPOSE:
// Add 2 numbers together.
// Illustrate use of files with streams.

#include <iostream>
#include <fstream>

int main(void)
{
    char input_filename[1024]; // Name of
                                // input file.
    char output_filename[1024]; // Name of
                                // output file.
    int a,b;                    // 2 ints from
                                // input file.
    int sum;                    // Sum of a and b.

    cout << "Enter the input file name ";
    cin >> input_filename;
    cout << "Enter the output filename ";
    cin >> output_filename;

    // Stream for input
    ifstream data_in(input_filename);

    // Stream for output
    ofstream data_out(output_filename);

    data_in >> a;
    data_in >> b;
    sum = a + b;
    data_out << "The sum of " << a << " and " << b
              << " is " << sum << endl;
    data_in.close();
    data_out.close();

    return 0;
}
```

Contents of file adder.in:

```
123
456
```

Output from adder:

```
Enter the input file name    adder.in
Enter the output filename   adder.out
```

Contents of file adder.out:

```
The sum of 123 and 456 is 579
```

The following small program illustrates how to recover from erroneous input:

Handling I/O errors

```
// FILENAME:   errin.cpp
// PROGRAMMER: Gary J. Blair
// DATE:       8/11/1996
// COMPILER:   MS Visual C++ 2008 Express Edition
// REQUIRED:    errin.cpp
// PURPOSE:
//   Illustrate recovery from stream errors.

#include <iostream>

using namespace std;

int main(void)
{
    int    i;           // Value to be input.
    int    success = 0;  // Have we succeeded yet?

    do
    {
        cout << "Enter an integer  ";
        cin >> i;

        if (cin.fail())    // Error inputting info
        {
            cout << "cin.fail() is true\n";
            cin.clear();    // Clear error flag.

            //Consume offending input characters
            cin.ignore(1024, '\n');

        }
        else
            success = 1;
    }
}
```

```
while (! success);  
cout << "Value of i=" << i << endl;  
  
return 0;  
}
```

Output from errin.cpp:

```
Enter an integer  junk  
cin.fail() is true  
Enter an integer  more stuff  
cin.fail() is true  
Enter an integer  23  
Value of i=23
```

Topic 3.7 (<https://onlinelearning.berkeley.edu/courses/2291660/pages/topic-3-dot-7>)